

Product Extractor Generator

Autocoder User Guide

1 Description

The purpose of this generator is to provide a user the ability to extract data from packet and create data products out of them. The generator takes a YAML model file as an input which is used by the component to determine which packets to extract data products from as well as the offset and type in the packet the component needs to extract. This is performed through an Ada specification and implementation that is autcoded to perform the extraction and verification of the data product. In the case that extracting a data product is invalid for the specified type, then an invalid data structure is returned with the value of the invalid product.

Note the example shown in this documentation is used in the unit test of this component so that the reader of this document can see it being used in context. Please refer to the unit test code for more details on how this generator can be used.

2 Schema

The following pykwalify schema is used to validate the input YAML model. The schema is commented to show what each of the available YAML keys are and what they accomplish. Even without knowing the specifics of pykwalify schemas, you should be able to glean some knowledge from the file below.

```
1  ---
2  # This schema describes the yaml format for a product extractor list.
3  type: map
4  mapping:
5    # Description of the data products from all packets to be extracted.
6    description:
7      type: str
8      required: False
9    # List of data products to include in the suite.
10   data_products:
11     seq:
12       - type: map
13         mapping:
14           # Name of the product.
15           name:
16             type: str
17             required: True
18           # Description of the data_products item.
19           description:
20             type: str
21             required: False
22           # Type of the product.
23           type:
24             type: str
25             required: True
26           # Apid of the product in the associated packet.
```

```

27     apid:
28         type: int
29         required: True
30     # Offset of the product in the associated packet.
31     offset:
32         type: int
33         required: True
34     # Time format of the packet.
35     time:
36         type: str
37         enum: ['current_time', 'packet_time']
38         required: True
39     # Optional default value for the product, supplied as an Ada aggregate
40     # string for the product's packed type (e.g. "(Value => 0)"). When
41     # present, the product extractor seeds this value into the data
42     ↪ product
43     # database during Set_Up, guaranteeing the product exists before the
44     # first packet arrives. The string is validated against the product
45     ↪ type
46     # by the Ada compiler. When omitted, no default is seeded (legacy
47     # behavior, byte-identical generated output).
48     default_at_set_up:
49         type: str
50         required: False
51     # A data product extractor must have at least one product to extract.
52     range:
53         min: 1
54         required: True

```

3 Example Input

The following is an example extracted product input yaml file. Model files must be named in the form *assembly_name.extracted_products.yaml* where the specific name of the extracted products is not allowed. The *assembly_name* is the assembly which this table will be used, and the rest of the model file name must remain as shown. Generally this file is created in the same directory or near to the assembly model file. This example adheres to the schema shown in the previous section, and is commented to give clarification.

```

1  ---
2  data_products:
3  - name: Test_Product_1
4    type: Packed_U16.T
5    apid: 100
6    offset: 10
7    time: packet_time
8  - name: Test_Product_2
9    type: Packed_Byte.T
10   apid: 100
11   offset: 16
12   time: current_time
13  - name: Test_Product_3
14    type: Packed_U32.T
15    apid: 200
16    offset: 8
17    time: current_time
18    default_at_set_up: "(Value => 42)"
19  - name: Test_Product_4
20    type: Packed_Natural.T

```

```

21     apid: 200
22     offset: 20
23     time: packet_time
24 -   name: Test_Product_5
25     description: Test product for the little endian version of a packed type.
26     type: Packed_Natural.T_Le
27     apid: 300
28     offset: 15
29     time: packet_time
30     default_at_set_up: "(Value => 7)"

```

The specified data products consist of everything that the user would normally be required to include when defining a data product for a component, with the addition of the offset in the packet and corresponding APID. The type is also required which must be in the form of a packed type. The last required field is the time type which is either the current time of extraction from the packet or the time contained in the packet.

4 Seeding Default Values at Set Up

Because the component only produces a data product when a matching packet is received, a product has no value in the data product database until its first packet arrives. Downstream consumers that fetch one of these products as a data dependency would therefore receive a `Not_Available` status at start up. To close this gap, a product may optionally specify a `default_at_set_up` value in its model entry. This is an Ada aggregate string for the product's packed type (for example `"(Value => 0)"` for a `Packed_U32.T`). The string is dropped verbatim into the generated code and is therefore validated against the product type by the Ada compiler.

When a product declares a `default_at_set_up`, the generator emits a builder function for it and attaches that builder to the product's entry in the extraction list. During the component's `Set_Up` procedure — which runs after all components are initialized but before the rate groups begin ticking — the component publishes one data product per declared default, stamped with the current system time. This guarantees a valid value exists in the database for every seeded product before the first packet is processed. Products that do not declare a default are not seeded, so models that omit the field retain the legacy behavior. The feature requires no change to the component's `Init` signature: the optional builder travels alongside the extraction function in the same list already passed to `Init`.

5 Example Output

The example input shown in the previous section produces the following Ada output. The `Data_Product_Extraction_List` variable should be passed into the CCSDS Product Extractor component's `Init` procedure during assembly initialization.

The main job of the generator here was to verify the input YAML for validity and then to translate the data to an Ada data structure and Ada extraction and verification function for each product for use by the component. The generator also dynamically creates the data products for the component based on the YAML input.

Ads file:

```

1  -- Standard includes:
2  with Product_Extractor_Types; use Product_Extractor_Types;
3  with Data_Product;
4  with Data_Product_Types; use Data_Product_Types;
5  with Ccsds_Space_Packet;
6  with Invalid_Product_Data;
7  with Sys_Time;

```

```

8
9 package Test_Products is
10
11     function Extract_And_Validate_Test_Product_1 (Pkt : in Ccsds_Space_Packet.T;
12     ↪ Id_Base : in Data_Product_Types.Data_Product_Id; Timestamp : in
13     ↪ Sys_Time.T; Dp : out Data_Product.T; Invalid_Data_Product : out
14     ↪ Invalid_Product_Data.T) return Product_Status;
15
16     function Extract_And_Validate_Test_Product_2 (Pkt : in Ccsds_Space_Packet.T;
17     ↪ Id_Base : in Data_Product_Types.Data_Product_Id; Timestamp : in
18     ↪ Sys_Time.T; Dp : out Data_Product.T; Invalid_Data_Product : out
19     ↪ Invalid_Product_Data.T) return Product_Status;
20
21     function Extract_And_Validate_Test_Product_3 (Pkt : in Ccsds_Space_Packet.T;
22     ↪ Id_Base : in Data_Product_Types.Data_Product_Id; Timestamp : in
23     ↪ Sys_Time.T; Dp : out Data_Product.T; Invalid_Data_Product : out
24     ↪ Invalid_Product_Data.T) return Product_Status;
25
26     function Extract_And_Validate_Test_Product_4 (Pkt : in Ccsds_Space_Packet.T;
27     ↪ Id_Base : in Data_Product_Types.Data_Product_Id; Timestamp : in
28     ↪ Sys_Time.T; Dp : out Data_Product.T; Invalid_Data_Product : out
29     ↪ Invalid_Product_Data.T) return Product_Status;
30
31     -- Test product for the little endian version of a packed type.
32     function Extract_And_Validate_Test_Product_5 (Pkt : in Ccsds_Space_Packet.T;
33     ↪ Id_Base : in Data_Product_Types.Data_Product_Id; Timestamp : in
34     ↪ Sys_Time.T; Dp : out Data_Product.T; Invalid_Data_Product : out
35     ↪ Invalid_Product_Data.T) return Product_Status;
36
37     -- Builders for the default ("seed") values of products that requested a
38     -- "default_at_set_up" in the model. The component publishes these during
39     -- Set_Up so the products exist in the data product database before the first
40     -- packet is processed.
41     function Make_Default_Test_Product_3 (Id_Base : in
42     ↪ Data_Product_Types.Data_Product_Id; Timestamp : in Sys_Time.T) return
43     ↪ Data_Product.T;
44     function Make_Default_Test_Product_5 (Id_Base : in
45     ↪ Data_Product_Types.Data_Product_Id; Timestamp : in Sys_Time.T) return
46     ↪ Data_Product.T;
47     Extract_Products_100 : aliased Extractor_List := [
48     0 => (Extract => Extract_And_Validate_Test_Product_1'Access, Make_Default
49     ↪ => null),
50     1 => (Extract => Extract_And_Validate_Test_Product_2'Access, Make_Default
51     ↪ => null)
52 ];
53     Extract_Products_200 : aliased Extractor_List := [
54     0 => (Extract => Extract_And_Validate_Test_Product_3'Access, Make_Default
55     ↪ => Make_Default_Test_Product_3'Access),
56     1 => (Extract => Extract_And_Validate_Test_Product_4'Access, Make_Default
57     ↪ => null)
58 ];
59     Extract_Products_300 : aliased Extractor_List := [
60     0 => (Extract => Extract_And_Validate_Test_Product_5'Access, Make_Default
61     ↪ => Make_Default_Test_Product_5'Access)
62 ];
63
64     -- Initial product extraction list containing the information to extract all
65     ↪ the products requested by each apid
66     Data_Product_Extraction_List : aliased Extracted_Product_List := [
67     0 => (
68     Apid => 100,

```

```

44     Extract_List => Extract_Products_100'Access
45 ),
46 1 => (
47     Apid => 200,
48     Extract_List => Extract_Products_200'Access
49 ),
50 2 => (
51     Apid => 300,
52     Extract_List => Extract_Products_300'Access
53 )
54 ];
55 Data_Product_Extraction_List_Access : constant Extracted_Product_List_Access
56   ↪ := Data_Product_Extraction_List'Access;
57 end Test_Products;

```

Adb file:

```

1  -- Standard includes:
2  with Basic_Types;
3  with Byte_Array_Util;
4  with Ccsds_Primary_Header; use Ccsds_Primary_Header;
5  with Interfaces; use Interfaces;
6  with Extract_Data_Product; use Extract_Data_Product;
7  with Packed_U16;
8  with Packed_U16.Validation;
9  with Packed_Byte;
10 with Packed_Byte.Validation;
11 with Packed_U32;
12 with Packed_U32.Validation;
13 with Packed_Natural;
14 with Packed_Natural.Validation;
15
16 package body Test_Products is
17
18     -- The implementation for each extract and validate of each extracted data
19     ↪ product
20 function Extract_And_Validate_Test_Product_1 (Pkt : in Ccsds_Space_Packet.T;
21 ↪ Id_Base : in Data_Product_Types.Data_Product_Id; Timestamp : in
22 ↪ Sys_Time.T; Dp : out Data_Product.T; Invalid_Data_Product : out
23 ↪ Invalid_Product_Data.T) return Product_Status is
24 Local_Id : constant Data_Product_Types.Data_Product_Id := 0;
25 Id : constant Data_Product_Types.Data_Product_Id := Id_Base + Local_Id;
26 Extraction_Status : Extract_Status;
27 Ignore : Sys_Time.T renames Timestamp;
28 begin
29 pragma Assert (Pkt.Header.Apid = 100);
30
31     -- Initialize out parameters:
32 Invalid_Data_Product := (
33     Id => Data_Product_Types.Data_Product_Id'First,
34     Errant_Field_Number => 0,
35     Errant_Field => [others => 0]
36 );
37
38     -- Use the generic extraction function using the autocoded values from the
39     ↪ YAML file to get the information needed from the packet
40 Extraction_Status := Extract_Data_Product.Extract_Data_Product (Pkt =>
41 ↪ Pkt, Offset => 10, Length => Packed_U16.Size_In_Bytes, Id => Id,
42 ↪ Timestamp => Sys_Time.Serialization.From_Byte_Array (Pkt.Data
43 ↪ (Pkt.Data'First .. Pkt.Data'First +
44 ↪ Sys_Time.Serialization.Serialized_Length - 1)), Dp => Dp);

```

```

36
37     -- Make sure the extraction was successful, at this point the only failure
38     ↪ should be the length
39     case Extraction_Status is
40         when Length_Overflow => return Length_Error;
41         when Success => null;
42     end case;
43
44     -- Now make sure the data product is valid for the type that we are
45     ↪ extracting using the validation from the packed type generation.
46     -- Validate directly on the byte buffer to avoid passing
47     ↪ potentially-invalid typed data.
48     declare
49         Ef : Unsigned_32;
50         Dp_Bytes : Packed_U16.Serialization.Byte_Array renames Dp.Buffer (
51             Dp.Buffer'First .. Dp.Buffer'First +
52             ↪ Packed_U16.Serialization.Byte_Array'Length - 1
53         );
54     begin
55         pragma Warnings (Off, "this code can never be executed and has been
56             ↪ deleted");
57         if Packed_U16.Always_Valid or else
58             Packed_U16.Validation.Valid (Bytes => Dp_Bytes, Errant_Field => Ef)
59         then
60             return Success;
61         else
62             -- When there is a validation error, fill in a data structure with
63             ↪ the relevant information for the component to use to send an
64             ↪ event.
65             declare
66                 P_Type : Basic_Types.Poly_Type := [others => 0];
67             begin
68                 -- Copy extracted value into poly type
69                 Byte_Array_Util.Safe_Right_Copy (P_Type, Pkt.Data (10 .. 10 +
70                     ↪ Packed_U16.Size_In_Bytes - 1));
71                 Invalid_Data_Product.Id := Id;
72                 Invalid_Data_Product.Errant_Field_Number := Ef;
73                 Invalid_Data_Product.Errant_Field := P_Type;
74             end;
75             return Invalid_Data;
76         end if;
77         pragma Warnings (On, "this code can never be executed and has been
78             ↪ deleted");
79     end;
80 end Extract_And_Validate_Test_Product_1;
81
82 function Extract_And_Validate_Test_Product_2 (Pkt : in Ccsds_Space_Packet.T;
83     ↪ Id_Base : in Data_Product_Types.Data_Product_Id; Timestamp : in
84     ↪ Sys_Time.T; Dp : out Data_Product.T; Invalid_Data_Product : out
85     ↪ Invalid_Product_Data.T) return Product_Status is
86     Local_Id : constant Data_Product_Types.Data_Product_Id := 1;
87     Id : constant Data_Product_Types.Data_Product_Id := Id_Base + Local_Id;
88     Extraction_Status : Extract_Status;
89 begin
90     pragma Assert (Pkt.Header.Apid = 100);
91
92     -- Initialize out parameters:
93     Invalid_Data_Product := (
94         Id => Data_Product_Types.Data_Product_Id'First,
95         Errant_Field_Number => 0,
96         Errant_Field => [others => 0]
97     );
98 end;

```

```

85     );
86
87     -- Use the generic extraction function using the autocoded values from the
88     -- ↪ YAML file to get the information needed from the packet
89     Extraction_Status := Extract_Data_Product.Extract_Data_Product (Pkt =>
90     -- ↪ Pkt, Offset => 16, Length => Packed_Byte.Size_In_Bytes, Id => Id,
91     -- ↪ Timestamp => Timestamp, Dp => Dp);
92
93     -- Make sure the extraction was successful, at this point the only failure
94     -- ↪ should be the length
95     case Extraction_Status is
96     when Length_Overflow => return Length_Error;
97     when Success => null;
98     end case;
99
100    -- Now make sure the data product is valid for the type that we are
101    -- ↪ extracting using the validation from the packed type generation.
102    -- Validate directly on the byte buffer to avoid passing
103    -- ↪ potentially-invalid typed data.
104    declare
105        Ef : Unsigned_32;
106        Dp_Bytes : Packed_Byte.Serialization.Byte_Array renames Dp.Buffer (
107        Dp.Buffer'First .. Dp.Buffer'First +
108        -- ↪ Packed_Byte.Serialization.Byte_Array'Length - 1
109        );
110    begin
111        pragma Warnings (Off, "this code can never be executed and has been
112        -- ↪ deleted");
113        if Packed_Byte.Always_Valid or else
114        Packed_Byte.Validation.Valid (Bytes => Dp_Bytes, Errant_Field => Ef)
115        then
116            return Success;
117        else
118            -- When there is a validation error, fill in a data structure with
119            -- ↪ the relevant information for the component to use to send an
120            -- ↪ event.
121            declare
122                P_Type : Basic_Types.Poly_Type := [others => 0];
123            begin
124                -- Copy extracted value into poly type
125                Byte_Array_Util.Safe_Right_Copy (P_Type, Pkt.Data (16 .. 16 +
126                -- ↪ Packed_Byte.Size_In_Bytes - 1));
127                Invalid_Data_Product.Id := Id;
128                Invalid_Data_Product.Errant_Field_Number := Ef;
129                Invalid_Data_Product.Errant_Field := P_Type;
130            end;
131            return Invalid_Data;
132        end if;
133        pragma Warnings (On, "this code can never be executed and has been
134        -- ↪ deleted");
135    end;
136end Extract_And_Validate_Test_Product_2;
137
138function Extract_And_Validate_Test_Product_3 (Pkt : in Ccsds_Space_Packet.T;
139-- ↪ Id_Base : in Data_Product_Types.Data_Product_Id; Timestamp : in
140-- ↪ Sys_Time.T; Dp : out Data_Product.T; Invalid_Data_Product : out
141-- ↪ Invalid_Product_Data.T) return Product_Status is
142    Local_Id : constant Data_Product_Types.Data_Product_Id := 2;
143    Id : constant Data_Product_Types.Data_Product_Id := Id_Base + Local_Id;
144    Extraction_Status : Extract_Status;
145begin

```

```

131     pragma Assert (Pkt.Header.Apid = 200);
132
133     -- Initialize out parameters:
134     Invalid_Data_Product := (
135         Id => Data_Product_Types.Data_Product_Id'First,
136         Errant_Field_Number => 0,
137         Errant_Field => [others => 0]
138     );
139
140     -- Use the generic extraction function using the autocoded values from the
141     -- ↪ YAML file to get the information needed from the packet
142     Extraction_Status := Extract_Data_Product.Extract_Data_Product (Pkt =>
143     -- ↪ Pkt, Offset => 8, Length => Packed_U32.Size_In_Bytes, Id => Id,
144     -- ↪ Timestamp => Timestamp, Dp => Dp);
145
146     -- Make sure the extraction was successful, at this point the only failure
147     -- ↪ should be the length
148     case Extraction_Status is
149     when Length_Overflow => return Length_Error;
150     when Success => null;
151     end case;
152
153     -- Now make sure the data product is valid for the type that we are
154     -- ↪ extracting using the validation from the packed type generation.
155     -- Validate directly on the byte buffer to avoid passing
156     -- ↪ potentially-invalid typed data.
157     declare
158         Ef : Unsigned_32;
159         Dp_Bytes : Packed_U32.Serialization.Byte_Array renames Dp.Buffer (
160             Dp.Buffer'First .. Dp.Buffer'First +
161             -- ↪ Packed_U32.Serialization.Byte_Array'Length - 1
162         );
163     begin
164         pragma Warnings (Off, "this code can never be executed and has been
165         -- ↪ deleted");
166         if Packed_U32.Always_Valid or else
167             Packed_U32.Validation.Valid (Bytes => Dp_Bytes, Errant_Field => Ef)
168         then
169             return Success;
170         else
171             -- When there is a validation error, fill in a data structure with
172             -- ↪ the relevant information for the component to use to send an
173             -- ↪ event.
174             declare
175                 P_Type : Basic_Types.Poly_Type := [others => 0];
176             begin
177                 -- Copy extracted value into poly type
178                 Byte_Array_Util.Safe_Right_Copy (P_Type, Pkt.Data (8 .. 8 +
179                 -- ↪ Packed_U32.Size_In_Bytes - 1));
180                 Invalid_Data_Product.Id := Id;
181                 Invalid_Data_Product.Errant_Field_Number := Ef;
182                 Invalid_Data_Product.Errant_Field := P_Type;
183             end;
184             return Invalid_Data;
185         end if;
186         pragma Warnings (On, "this code can never be executed and has been
187         -- ↪ deleted");
188     end;
189 end Extract_And_Validate_Test_Product_3;
190
191 function Extract_And_Validate_Test_Product_4 (Pkt : in Ccsds_Space_Packet.T;
192 -- ↪ Id_Base : in Data_Product_Types.Data_Product_Id; Timestamp : in
193 -- ↪ Sys_Time.T; Dp : out Data_Product.T; Invalid_Data_Product : out
194 -- ↪ Invalid_Product_Data.T) return Product_Status is

```



```

180     Local_Id : constant Data_Product_Types.Data_Product_Id := 3;
181     Id : constant Data_Product_Types.Data_Product_Id := Id_Base + Local_Id;
182     Extraction_Status : Extract_Status;
183     Ignore : Sys_Time.T renames Timestamp;
184 begin
185     pragma Assert (Pkt.Header.Apid = 200);
186
187     -- Initialize out parameters:
188     Invalid_Data_Product := (
189         Id => Data_Product_Types.Data_Product_Id'First,
190         Errant_Field_Number => 0,
191         Errant_Field => [others => 0]
192     );
193
194     -- Use the generic extraction function using the autocoded values from the
195     -- ↪ YAML file to get the information needed from the packet
196     Extraction_Status := Extract_Data_Product.Extract_Data_Product (Pkt =>
197     -- ↪ Pkt, Offset => 20, Length => Packed_Natural.Size_In_Bytes, Id => Id,
198     -- ↪ Timestamp => Sys_Time.Serialization.From_Byte_Array (Pkt.Data
199     -- ↪ (Pkt.Data'First .. Pkt.Data'First +
200     -- ↪ Sys_Time.Serialization.Serialized_Length - 1)), Dp => Dp);
201
202     -- Make sure the extraction was successful, at this point the only failure
203     -- ↪ should be the length
204     case Extraction_Status is
205     when Length_Overflow => return Length_Error;
206     when Success => null;
207     end case;
208
209     -- Now make sure the data product is valid for the type that we are
210     -- ↪ extracting using the validation from the packed type generation.
211     -- Validate directly on the byte buffer to avoid passing
212     -- ↪ potentially-invalid typed data.
213     declare
214         Ef : Unsigned_32;
215         Dp_Bytes : Packed_Natural.Serialization.Byte_Array renames Dp.Buffer (
216             Dp.Buffer'First .. Dp.Buffer'First +
217             ↪ Packed_Natural.Serialization.Byte_Array'Length - 1
218         );
219     begin
220         pragma Warnings (Off, "this code can never be executed and has been
221         -- ↪ deleted");
222         if Packed_Natural.Always_Valid or else
223             Packed_Natural.Validation.Valid (Bytes => Dp_Bytes, Errant_Field =>
224             -- ↪ Ef)
225         then
226             return Success;
227         else
228             -- When there is a validation error, fill in a data structure with
229             -- ↪ the relevant information for the component to use to send an
230             -- ↪ event.
231             declare
232                 P_Type : Basic_Types.Poly_Type := [others => 0];
233             begin
234                 -- Copy extracted value into poly type
235                 Byte_Array_Util.Safe_Right_Copy (P_Type, Pkt.Data (20 .. 20 +
236                 -- ↪ Packed_Natural.Size_In_Bytes - 1));
237                 Invalid_Data_Product.Id := Id;
238                 Invalid_Data_Product.Errant_Field_Number := Ef;
239                 Invalid_Data_Product.Errant_Field := P_Type;
240             end;
241         end;
242     end;

```

```

227         return Invalid_Data;
228     end if;
229     pragma Warnings (On, "this code can never be executed and has been
        ↳ deleted");
230 end;
231 end Extract_And_Validate_Test_Product_4;
232
233 function Extract_And_Validate_Test_Product_5 (Pkt : in Ccsds_Space_Packet.T;
        ↳ Id_Base : in Data_Product_Types.Data_Product_Id; Timestamp : in
        ↳ Sys_Time.T; Dp : out Data_Product.T; Invalid_Data_Product : out
        ↳ Invalid_Product_Data.T) return Product_Status is
234     Local_Id : constant Data_Product_Types.Data_Product_Id := 4;
235     Id : constant Data_Product_Types.Data_Product_Id := Id_Base + Local_Id;
236     Extraction_Status : Extract_Status;
237     Ignore : Sys_Time.T renames Timestamp;
238 begin
239     pragma Assert (Pkt.Header.Apid = 300);
240
241     -- Initialize out parameters:
242     Invalid_Data_Product := (
243         Id => Data_Product_Types.Data_Product_Id'First,
244         Errant_Field_Number => 0,
245         Errant_Field => [others => 0]
246     );
247
248     -- Use the generic extraction function using the autocoded values from the
        ↳ YAML file to get the information needed from the packet
249     Extraction_Status := Extract_Data_Product.Extract_Data_Product (Pkt =>
        ↳ Pkt, Offset => 15, Length => Packed_Natural.Size_In_Bytes, Id => Id,
        ↳ Timestamp => Sys_Time.Serialization.From_Byte_Array (Pkt.Data
        ↳ (Pkt.Data'First .. Pkt.Data'First +
        ↳ Sys_Time.Serialization.Serialized_Length - 1)), Dp => Dp);
250
251     -- Make sure the extraction was successful, at this point the only failure
        ↳ should be the length
252     case Extraction_Status is
253         when Length_Overflow => return Length_Error;
254         when Success => null;
255     end case;
256
257     -- Now make sure the data product is valid for the type that we are
        ↳ extracting using the validation from the packed type generation.
258     -- Validate directly on the byte buffer to avoid passing
        ↳ potentially-invalid typed data.
259     declare
260         Ef : Unsigned_32;
261         Dp_Bytes : Packed_Natural.Serialization_Le.Byte_Array renames Dp.Buffer
        ↳ (
262             Dp.Buffer'First .. Dp.Buffer'First +
        ↳ Packed_Natural.Serialization_Le.Byte_Array'Length - 1
263         );
264     begin
265         pragma Warnings (Off, "this code can never be executed and has been
        ↳ deleted");
266         if Packed_Natural.Always_Valid or else
267             Packed_Natural.Validation.Valid_Le (Bytes => Dp_Bytes, Errant_Field
        ↳ => Ef)
268         then
269             return Success;
270         else
271             -- When there is a validation error, fill in a data structure with
        ↳ the relevant information for the component to use to send an
        ↳ event.

```

```

272         declare
273             P_Type : Basic_Types.Poly_Type := [others => 0];
274         begin
275             -- Copy extracted value into poly type
276             Byte_Array_Util.Safe_Right_Copy (P_Type, Pkt.Data (15 .. 15 +
                ↳ Packed_Natural.Size_In_Bytes - 1));
277             Invalid_Data_Product.Id := Id;
278             Invalid_Data_Product.Errant_Field_Number := Ef;
279             Invalid_Data_Product.Errant_Field := P_Type;
280         end;
281         return Invalid_Data;
282     end if;
283     pragma Warnings (On, "this code can never be executed and has been
        ↳ deleted");
284 end;
285 end Extract_And_Validate_Test_Product_5;
286
287 -- Build the default ("seed") value for Test_Product_3. The default
288 -- value string from the model is validated against the product type by the
289 -- compiler.
290 function Make_Default_Test_Product_3 (Id_Base : in
    ↳ Data_Product_Types.Data_Product_Id; Timestamp : in Sys_Time.T) return
    ↳ Data_Product.T is
291     Local_Id : constant Data_Product_Types.Data_Product_Id := 2;
292     Id : constant Data_Product_Types.Data_Product_Id := Id_Base + Local_Id;
293     Dp : Data_Product.T := (
294         Header => (Time => Timestamp, Id => Id, Buffer_Length =>
            ↳ Packed_U32.Serialization.Serialized_Length),
295         Buffer => [others => 0]
296     );
297 begin
298     Dp.Buffer (Dp.Buffer'First .. Dp.Buffer'First +
        ↳ Packed_U32.Serialization.Serialized_Length - 1) :=
299         Packed_U32.Serialization.To_Byte_Array (Packed_U32.T'(Value => 42));
300     return Dp;
301 end Make_Default_Test_Product_3;
302
303 -- Build the default ("seed") value for Test_Product_5. The default
304 -- value string from the model is validated against the product type by the
305 -- compiler.
306 function Make_Default_Test_Product_5 (Id_Base : in
    ↳ Data_Product_Types.Data_Product_Id; Timestamp : in Sys_Time.T) return
    ↳ Data_Product.T is
307     Local_Id : constant Data_Product_Types.Data_Product_Id := 4;
308     Id : constant Data_Product_Types.Data_Product_Id := Id_Base + Local_Id;
309     Dp : Data_Product.T := (
310         Header => (Time => Timestamp, Id => Id, Buffer_Length =>
            ↳ Packed_Natural.Serialization_Le.Serialized_Length),
311         Buffer => [others => 0]
312     );
313 begin
314     Dp.Buffer (Dp.Buffer'First .. Dp.Buffer'First +
        ↳ Packed_Natural.Serialization_Le.Serialized_Length - 1) :=
315         Packed_Natural.Serialization_Le.To_Byte_Array
            ↳ (Packed_Natural.T_Le'(Value => 7));
316     return Dp;
317 end Make_Default_Test_Product_5;
318
319 end Test_Products;

```